Register Bombs: How Scope Processing Granularity Creates Catastrophic Instruction Interference in LLMs

Tony Mason
the University of British Columbia
the Georgia Institute of Technology
fsgeek@{cs.ubc.ca,gatech.edu,wamason.com}

March 2026

Abstract

A single imperative prohibition embedded in an otherwise declarative system prompt can collapse adherence to semantically unrelated instructions. We call this a *register bomb*: a block whose imperative register, when contrasted against a declarative context, causes the model to over-generalize its prohibitions. In Claude Haiku 4.5, making one commit-workflow block imperative ("NEVER use Task tools") while keeping ten other blocks declarative drops adherence to an unrelated explore-agent instruction from 1.000 to 0.200.

We isolate the mechanism through four experiments. First, the effect is block-specific, not a general sensitivity to register contrast (E-PHASE-CONFIRM). Second, scope must be *structurally embedded* in each prohibition — a block-level prefix ("During commits only:") fails to prevent the collapse (explore-agent = 0.167), while per-prohibition inline conditionals ("When committing, NEVER use...") fully rescue it (1.000). This reveals that models process register at **clause granularity**, not block granularity. Third, rescue and re-collapse dynamics across eleven density conditions show that specific block pairs create constructive and destructive interference patterns. Fourth, cross-model replication reveals a probe battery transfer problem: model-specific instruction formats create floor effects that confound cross-architecture comparison.

These findings establish that prompt engineering must attend not only to *what* instructions say but to the *syntactic proximity* of scope markers to prohibitions, and that register uniformity — or careful scope embedding — is necessary to prevent catastrophic interference between instructions that share no semantic relationship.

We demonstrate the effect in Claude Haiku 4.5; cross-model replication is confounded by probe transfer, and the generality of the phenomenon is an open question.

1 Introduction

System prompts for deployed LLM agents contain dozens to hundreds of instructions that must be followed simultaneously. Prior work has shown that these instructions interact — cooperating or competing depending on language, model, and speech act type [Mason, 2026a,b]. The social register framework established that imperative instructions ("NEVER do X") carry different obligatory force than declarative equivalents ("X: disabled"), and that this difference is magnified across languages.

This paper investigates what happens at the boundary between registers: when a small number of imperative instructions are embedded in an otherwise declarative prompt. We discover a failure mode we term a *register bomb* — a single mismatched-register block that causes catastrophic interference with semantically unrelated instructions. The mechanism is *scope loss*:

an imperative prohibition scoped to a specific workflow (git commits) is over-generalized to all model behavior when it lacks register context from neighboring imperatives.

Register bomb. An instruction block B in register R_1 embedded in a prompt otherwise written in register R_2 ($R_1 \neq R_2$), where B's inclusion causes adherence collapse on instructions semantically unrelated to B. The collapse is mediated by register contrast, not semantic overlap.

Our central finding is that scope interpretation operates at **clause granularity**. A block-level scope declaration ("During git commit workflows only:") does not propagate to constituent prohibitions. Each imperative clause is processed independently for its obligatory force. This has a direct practical consequence: scope must be *embedded in* each prohibition ("When committing, NEVER use Task tools"), not declared above it.

1.1 Contributions

1. **Register bombs:** We identify and isolate a catastrophic interference pattern where a single imperative block in a declarative context collapses adherence to unrelated instructions by up to 80%.

2. **Clause-level scope processing:** We demonstrate that models process register and scope at clause granularity, not block granularity, through a controlled comparison of prefix vs. inline scoping interventions.

3. **Interaction dynamics:** We characterize rescue and re-collapse patterns across eleven density conditions, showing that register interference is pairwise and non-monotonic.

4. **Probe transfer problem:** We document a methodological limitation for cross-model register studies: instruction-format-specific probe batteries create floor effects on non-target models.

5. **Design principles:** We derive actionable guidelines for system prompt engineering: embed scope per-prohibition, maintain register uniformity, and test for register bombs when mixing instruction styles.

2 Background

2.1 Social Register in LLM Instructions

Mason [2026b] established that instruction adherence in LLMs is mediated by social register — the speech act type of the instruction (imperative command vs. declarative statement) rather than its semantic content. Cross-linguistic experiments showed that the same instruction in imperative form cooperates in English but competes in Spanish, and that declarative rewriting eliminates this topology inversion with 81% variance reduction.

That work treated register as a property of individual blocks. This paper examines what happens at register *boundaries*: the interface between blocks written in different registers within a single prompt.

2.2 Scope in Natural Language

Scope ambiguity is a well-studied phenomenon in formal semantics [May, 1977]. Quantifiers, negation, and modal operators can take wide or narrow scope depending on syntactic structure. Our finding that LLMs resolve scope similarly — treating each clause as self-contained unless structural embedding forces a different reading — suggests that models have learned scope resolution patterns from their training data rather than implementing explicit scope tracking.

2.3 Instruction Interference

Prior work on prompt sensitivity has shown that small perturbations to instructions can cause large behavioral changes [Prasad et al., 2025]. However, most work treats instructions as independent units. Mason [2026a] introduced pairwise ablation to measure instruction interactions, finding that interference is the norm rather than the exception. Register bombs are a specific, high-severity instance of this general phenomenon.

3 Experimental Setup

3.1 Corpus and Probe Battery

We use the Claude Code v2.1.50 system prompt, containing 56 blocks of which 22 are individually ablatable ("free blocks") and 11 are procedural/imperative in their original form. We maintain the probe battery from Mason [2026b]: 22 hand-authored probes, each scored via `not_contains`, `length`, or `llm_judge` methods, with 3 trials per condition at temperature 0.0.

3.2 Declarative Rewrites

Each of the 11 procedural blocks has a declarative equivalent that preserves semantic content while replacing imperative mood with factual statements. For example, the commit-restrictions block transforms from:

> "NEVER use the TodoWrite or Task tools"
> "DO NOT push to the remote repository unless..."

to:

> "Disallowed tools: TodoWrite, Task"
> "Push policy: requires explicit user request"

3.3 Models

Primary experiments use Claude Haiku 4.5 via OpenRouter. Cross-model replication uses Gemini Flash 2.0, DeepSeek v3 (0324), and Mistral Medium 3.1. All API calls use temperature 0.0.

3.4 Cost

Total experimental cost across all experiments reported in this paper: $5.70 (approximately 5,400 API calls including judge calls).

4 Experiment 1: Register Bomb Isolation

4.1 Design

E-PHASE mapped adherence across 12 density conditions (0 = all declarative through 11 = all imperative). When density increased from 0 to 1 — adding *only* the commit-restrictions block as imperative — explore-agent adherence collapsed from 1.000 to 0.200. E-PHASE-CONFIRM tested whether this was block-specific or a general "lone wolf" effect.

Six conditions:

Condition	Description	explore-agent	proactive
all-decl	All declarative	1.000	0.783
only-cr-imp	Only CR imperative	**0.200**	0.150
only-ea-imp	Only EA imperative	1.000	0.833
only-tw-imp	Only TW imperative	0.983	0.750
all-except-cr	All imp except CR decl	1.000	0.817
all-imp	All imperative	1.000	0.850

Table 1: E-PHASE-CONFIRM: Only commit-restrictions as a lone imperative collapses explore-agent. Other lone imperatives do not.

4.2 Results

The collapse is block-specific: only commit-restrictions triggers it (explore-agent = 0.200). Explore-agent and todowrite as lone imperatives cause no collapse (1.000, 0.983). Making commit-restrictions the lone *declarative* in an imperative field causes no collapse (all-except-cr = 1.000). The effect requires three conditions: (1) commit-restrictions is imperative, (2) surrounding blocks are declarative, and (3) there is a register contrast.

The affected probes — explore-agent, proactive-agents, use-task-for-search — are all **tool-delegation behaviors**. Commit-restrictions is about **git workflow constraints**. There is no semantic connection. The interference is mediated by register contrast, not semantic overlap.

5 Experiment 2: Scope Embedding

5.1 Hypothesis

The scope hypothesis: commit-restrictions contains "NEVER use the TodoWrite or Task tools," which is scoped to commit workflows but reads as a universal prohibition when register-isolated. Explicit scoping should prevent the collapse.

5.2 Design

Three conditions, each with commit-restrictions as the lone non-declarative block:

1. **scoped-prefix**: Original imperative text prefixed with "During git commit workflows only — the following restrictions apply exclusively when creating commits, not during other tasks:"
2. **scoped-inline**: Each prohibition rewritten as a conditional: "When committing, NEVER use the TodoWrite or Task tools (these tools remain available for non-commit tasks)"
3. **hybrid-decl-never**: Declarative list format but keeping "NEVER" on the key prohibition: "NEVER use the TodoWrite or Task tools during commits"

5.3 Results

Condition	explore-agent	proactive	use-task
all-decl (baseline)	1.000	0.783	0.500
only-cr-imp (baseline)	0.200	0.150	0.000
scoped-prefix	**0.167**	0.717	1.000
scoped-inline	**1.000**	0.850	0.983
hybrid-decl-never	**1.000**	0.667	0.500

Table 2: E-SCOPE: Block-level scope prefix fails (0.167). Per-prohibition inline scope succeeds (1.000). Scope must be structurally embedded.

5.4 Analysis

The scope hypothesis is wrong as stated. A prefix declaring scope does not prevent the collapse (0.167, indistinguishable from the unscoped 0.200). But inline conditionals fully rescue it (1.000).

This reveals the granularity of scope processing: the model does not propagate a scope-setting prefix to downstream imperative clauses. Each clause is processed independently for its register signal and obligatory force. "During commits only:" is metadata; "NEVER use Task tools" is a command. The prefix does not change the register of the command.

Inline conditionals work because they embed scope *in* the prohibition: "When committing, NEVER use Task tools" is a different speech act from "NEVER use Task tools." The former is a conditional command; the latter is unconditional.

The hybrid-decl-never condition (1.000) confirms that the word "NEVER" itself is not the trigger — it is the register context in which "NEVER" appears. In a declarative list, "NEVER" is contained by the list structure.

5.4.1 Secondary finding: differential thresholds

Scoped-prefix partially rescues proactive-agents (0.717 vs. 0.150 baseline) while failing to rescue explore-agent (0.167 vs. 0.200). Different target probes have different interference thresholds. The prefix provides enough disambiguation for weaker interference but not for the strongest pair.

6 Experiment 3: Interaction Dynamics

6.1 Density Trajectory

E-PHASE varied imperative density from 0 (all declarative) to 11 (all imperative), adding one block at a time ordered by cross-linguistic variance. The explore-agent trajectory reveals rescue and re-collapse dynamics:

Density	Block added	explore-agent	Pattern
0	(all declarative)	1.000	baseline
1	commit-restrictions	0.200	BOMB
2–5	(various)	0.150–0.383	collapsed
6	explore-agent	0.850	SELF-RESCUE
7	pr-workflow	0.167	RE-COLLAPSE
8	commit-workflow	0.267	collapsed
9	todowrite	1.000	FULL RESCUE
10	no-overengineering	0.217	RE-COLLAPSE
11	dedicated-tools	1.000	RESCUE

Table 3: Explore-agent adherence across density conditions showing non-monotonic rescue/collapse dynamics.

6.2 Interpretation

The trajectory is non-monotonic: adherence does not degrade smoothly with increasing imperative density. Instead, specific blocks create constructive interference (rescue) or destructive interference (re-collapse). Making explore-agent's own block imperative partially rescues it (d6 = 0.850) — the model recognizes the instruction in its native register. But adding pr-workflow (d7) re-collapses it, suggesting that pr-workflow's imperative form creates new interference.

The full rescue at density 9 (todowrite) and re-collapse at density 10 (no-overengineering) followed by rescue at density 11 (dedicated-tools) shows that the interaction structure is pairwise, not aggregate. You cannot predict adherence from density alone — you must know *which* blocks are imperative.

7 Experiment 4: Cross-Model Replication

7.1 Design

We tested three conditions (all-decl, only-cr-imp, scoped-inline) on three additional models: Gemini Flash 2.0, DeepSeek v3, and Mistral Medium 3.1.

7.2 Results

Model	all-decl	only-cr-imp	scoped-inline	Bomb?
Haiku 4.5	1.000	**0.200**	1.000	Yes
Gemini Flash	1.000	1.000	1.000	No
DeepSeek v3	0.333	0.667	0.667	No*
Mistral Med. 3.1	0.167	0.500	0.333	No*

Table 4: Cross-model bomb detonation matrix. *Floor baseline invalidates measurement.

7.3 The Probe Transfer Problem

The bomb detonates only on Haiku (1/4 models). However, the negative results on DeepSeek and Mistral are confounded: their all-decl baselines (0.333, 0.167) show that the probe battery does not transfer to models not trained on similar instruction formats. We cannot distinguish "model lacks register sensitivity" from "probes are invalid for this model."

Gemini's immunity (1.000 across all conditions) is more informative but still ambiguous: it may reflect genuine register robustness, different instruction processing, or probe insensitivity to the manipulation.

Methodological implication: Cross-model register studies require model-agnostic probe batteries designed around standardized tasks rather than model-specific instruction formats.

8 Discussion

8.1 Why Clause Granularity?

Our finding that scope is processed at clause rather than block granularity is consistent with how scope ambiguity is resolved in natural language. In formal semantics, scope is determined by syntactic structure, not by discourse-level declarations [May, 1977]. A speaker who says "In the kitchen: never touch the stove, never open the oven" creates two prohibitions scoped to the kitchen. But the scoping depends on the listener maintaining the discourse context — something humans do reliably but, evidently, current LLMs do not when the discourse marker ("in the kitchen") is separated from the prohibition by enough intervening text or register shift.

The practical consequence is that prompt engineers cannot rely on header-level scope declarations. Each prohibition must carry its own scope syntactically, either through conditional clauses ("When X, never Y") or through structural containment (declarative lists).

8.2 The Mechanistic Gap

We characterize the phenomenon but do not explain the internal mechanism. We observe *what* happens — scope loss at clause granularity — but not *why* the model behaves this way when the bomb detonates. At least three candidate mechanisms are consistent with our data. (1) **Unconditional-command parsing:** the imperative block's prohibitions ("NEVER use Task tools") are parsed as unconditional commands carrying higher authority than the surrounding declarative instructions, suppressing anything that resembles tool delegation. (2) **Register-shift-as-emphasis:** the register contrast is itself a signal — the model interprets the shift as emphasis, and the emphasis bleeds to semantically adjacent behaviors (tool use $\rightarrow$ tool delegation). (3) **No scope stack:** the model's instruction-following mechanism does not maintain a scope stack, so each clause is evaluated against the full instruction set independently. Distinguishing these candidates requires access to model internals — attention analysis, intermediate-layer probing, or targeted ablation of the model itself rather than the prompt — which is out of scope for this paper. We leave the mechanistic account as an open question.

8.3 Register Bombs as a Design Hazard

Register bombs arise naturally when prompt authors mix instruction styles — a common practice when system prompts are assembled from multiple sources or authored incrementally. The commit-restrictions block in Claude Code's system prompt was presumably written to be emphatic ("NEVER"), not to create interference with explore-agent instructions written in a different section by a different author.

This suggests that system prompt compilation should include register consistency checks: either enforce uniform register or verify that mixed-register blocks do not create pairwise interference. Our probe-based approach provides a testing methodology for this.

8.4 Limitations

1. **Single corpus:** All experiments use the Claude Code v2.1.50 system prompt. Register bombs in other prompts may have different characteristics.

2. **Single primary model:** The bomb, scope, and dynamics findings are from Haiku 4.5. Cross-model replication was confounded by probe transfer.

3. **Temperature 0.0:** All trials use deterministic decoding to reduce variance in a small-N study (3 trials per condition), isolating the register manipulation from sampling noise. Whether the effect persists under stochastic decoding (e.g. temperature 0.7) is left to future work; stochastic behavior may attenuate or amplify the effects.

4. **LLM-as-judge:** Nine of 22 probes use LLM judge scoring, which introduces scorer-dependent variance. We did not measure inter-judge agreement for this paper. As corroborating evidence, independent re-grading of LLM-judge outputs in a sister experiment found ∼86% two-judge agreement on outcome labels, with a third judge resolving 12/12 disagreements — indicating that LLM-judge disagreement on hard cases is largely recoverable variance rather than irreducible ambiguity, and motivating the use of ≥ 3 judges on boundary cases. We flag ensemble judging as the appropriate mitigation for the scorer-dependent variance in this study.

5. **Probe battery specificity:** The probe battery was designed for Claude Code and does not transfer to other models, limiting cross-model claims.

9 Related Work

Mason [2026a] introduced instruction-level ablation for system prompt analysis, identifying pairwise interference patterns in a 56-block production system prompt. Mason [2026b] extended this to show that interference topology is mediated by social register across languages, with declarative rewriting as a fix.

Prasad et al. [2025] demonstrated prompt sensitivity to small perturbations. Zhang et al. [2025] examined cross-lingual system prompt steerability but treated prompts as monolithic rather than analyzing instruction-level interactions.

Bulté and Rigouts Terryn [2025] showed that prompt language and cultural framing influence model responses, finding systematic bias toward specific cultural defaults. Our work extends this by showing that the *speech act type* of instructions, not just their language, determines interaction patterns.

To our knowledge, no prior work has identified register bombs, characterized clause-level scope processing in LLM instruction following, or documented the probe transfer problem for cross-model register studies.

10 Conclusion

Register bombs are a concrete, measurable failure mode in LLM system prompts. A single imperative prohibition, when register-isolated in a declarative context, can collapse adherence to unrelated instructions by up to 80%. The mechanism is scope loss at clause granularity: the model processes each prohibition independently, and a block-level scope declaration does not propagate to constituent clauses.

The fix is straightforward: embed scope in each prohibition ("When X, never Y") or use uniform register throughout. The broader implication is that system prompt engineering must attend to register interactions between instructions, not just the content of individual instructions. Prompt compilation tools should check for register consistency and test for pairwise interference. In a system where prompts are compiled from a typed instruction DSL with explicit scope annotations, register bombs would be caught at compile time as scope violations — a static guarantee that no amount of careful natural-language authoring can provide.

Acknowledgments

Experiments conducted using Claude Haiku 4.5, Gemini Flash 2.0, DeepSeek v3, and Mistral Medium 3.1 via the OpenRouter API. Total experimental cost: $5.70. Analysis and drafting assisted by Claude Opus 4 (Anthropic). The author thanks the anonymous reviewers of Mason [2026b] for feedback on the social register framework.

References

Bram Bulté and Ayla Rigouts Terryn. LLMs and cultural values: The impact of prompt language and explicit cultural framing. *Computational Linguistics*, pages 1–85, 2025.

Tony Mason. Arbiter: Detecting interference in LLM agent system prompts. *arXiv preprint arXiv:2603.08993*, 2026a.

Tony Mason. Imperative interference: Social register shapes instruction topology in large language models. *arXiv preprint*, 2026b. Under review.

Robert May. *The Grammar of Quantification*. PhD thesis, Massachusetts Institute of Technology, 1977.

Archiki Prasad et al. Prompt sensitivity in large language models. *arXiv preprint*, 2025.

Lechen Zhang, Yusheng Zhou, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. Cross-lingual prompt steerability: Towards accurate and robust llm behavior across languages. *arXiv preprint arXiv:2512.02841*, 2025.